

Last modified on

Sports Data Soccer API - In-Venue Player Shape Feed (MA51)

Get list of all the shapes and formations for a specific player in a team that has played in a match, based on “In-venue” data.

Overview

The Soccer **Player Shape in-Venue** feed provides the shape and formation of each player who had played in a match, based on “in venue” data.

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our **FAQs**. We also recommend that you take a look at our **API Overview and Authentication Guide**. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY							
	TIER 15	TIER 12+14	TIER 4	TIER 13	TIER 8+9+10+11	TIER 6+7	TIER 5	TIER 1+2+3
Updated on every stop in play during the game, including for the following types of events: goal, shot at goal, assist, free kick, offside given, cards, start of the half, end of the half, corner, substitution, lineup Note: There will always be a gap of at least 30 seconds between files unless a goal is scored	See Tier 13	Post-match	n/a	No more frequently than one URL poll every 30 seconds for range: <KO - 1h; KO + 4h> Up to 1-hour interval for range: <KO + 4h; KO + 8h> Up to 24-hour interval 8h+ of match ending, lasting a maximum of 7 days	n/a	n/a	n/a	n/a

Player Shape In-Venue (MA51) Guide

USAGE	FEE
GET information about shape and formation of each player associated with a specific MFL entity UUID, based on “in venue” data <i>Required:</i> query parameter and UUID for the entity (such as a fixture)	/s

If your request URL is as follows (does not contain query criteria), the feed returns an error:
/soccerdata/invenueplayershape/{outletAuthKey}

Query Parameters And Example Request Parameters

This feed also makes use of the **Global query parameters**, and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you must query the `/invenueplayershape` feed by an entity such as a fixture UUID and you can combine query parameters in your requests.

Legacy IDs: You can use the `fx` parameters to filter by legacy IDs.

How to use query parameters and make successful requests:
In our guides, we use HTTPS syntax for our example GET requests (rather than cURL, for example). All URIs are case-sensitive - this means that URL strings must contain the correct case and operators. You MUST format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the **main guide** and **API Overview and Authorisation guide**.

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` (Outlet Authentication Key), query parameters for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format), entity query parameter and UUID.
- Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or `-` (multiple values, 'OR') or `!` ('NOT').
- Use the required `_` (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces `{ }` from your calls (we include these as an example only).
- If using the JSONP format (`_fmt=jsonp`) you MUST use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
<code>{fixtureUuid}</code> Fixture UUID	Get In-venue player shape data by passing the fixture/match UUID inline URL path. GET https://api.performfeeds.com/soccerdata/invenueplayershape/{outletAuthKey}/{fixtureUuid?_fmt={dataFormat}&_rt={mode}
<code>fx</code>	GET In-venue player shape data by the specified fixture . Pass the match ID to the <code>fx</code> parameter.

Query parameter	Value and description
	GET https://api.performfeeds.com/soccerdata/invenueplayershape/{outletAuthKey}?fx={fixtureUuid}&_fmt={dataFormat}&_rt={mode}
fx Opta Fixture ID	GET In-venue player shape data by legacy Opta Id {fx query parameter method by (urn)}. Pass the Opta fixture ID to the fx parameter. GET https://api.performfeeds.com/soccerdata/invenueplayershape/{outletAuthKey}?fx={urn:perform:opta:Opta fixture Id}&_fmt={dataFormat}&_rt={mode}

Sample Calls (Coming Soon)

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

- ▶<inVenuePlayerShape> (XML only)
- ▶<matchInfo>
- ▶<description>
- ▶<sport>
- ▶<ruleset>
- ▶<competition>
- ▶<country>
- ▶<tournamentCalendar>
- ▶<stage>
- ▶<series>
- ▶<contestants> (XML only)
- ▶<contestant>
- ▶<country>
- ▶<venue>
- ▶<liveData>
- ▶<matchDetails>
- ▶<periods> (XML only)

▶<period>

▶<scores>

▶<ht>

▶<ft>

▶<total>

▶<goal>

▶<card>

▶<substitute>

▶<lineUp>

▶<teamOfficial>

▶<player>

▶<inPossession>

▶<shapeRole>

▶<role>

▶<outcome>

▶<outOfPossession>

▶<shapeRole>

▶<role>

▶<outcome>